

LABORATORIO DE BANCO DE DADOS

High Frequency Trades SIMULATOR

Order Book de Criptomoedas

Trabalho Final

Grupo	Até 3 alunos (recomendado)
Prazo	20/06/2026 até 23h59 - (sábado) não prorrogável
Banco de Dados	PostgreSQL 15+
Linguagem	Livre escolha do grupo
Volume de Dados	700 MB a 1 GB
Professor	Márcio Silva

... Conceitos reais usados por exchanges como Binance e Coinbase ...

1. Contexto

Em exchanges de criptomoedas reais, o núcleo do sistema é o **Order Book** — uma estrutura que organiza todas as intenções de compra e venda de um ativo. Quando os preços se encontram, uma transação é executada automaticamente, de forma atômica e sem inconsistências.

Este trabalho propõe construir um **simulador desse sistema inteiramente sustentado pelo PostgreSQL**, enfrentando desafios reais de concorrência, atomicidade e performance.

Bid (Compra): intenção de comprar um ativo por até um preço máximo.

Ask (Venda): intenção de vender um ativo por no mínimo um preço.

Match: ocorre quando $\text{bid_price} \geq \text{ask_price}$. A transação é executada automaticamente.

Partial Fill: uma ordem pode ser atendida parcialmente por múltiplas ordens opostas.

2. O que deve ser construído

2.1 Banco de Dados (PostgreSQL)

O grupo é responsável por **toda a modelagem**. Não há estrutura de tabelas fornecida — as decisões de design fazem parte da avaliação. O banco deve suportar:

- **Cadastro de usuários e carteiras** com controle de saldo disponível e saldo bloqueado por ordens abertas.
- **Order Book** com ordens de compra e venda por ativo, contendo estado (aberta, parcial, executada, cancelada).
- **Registro imutável de trades** executados — nenhum trade pode ser alterado ou removido após registrado.
- **Série temporal de preços** (candles OHLCV por minuto), particionada por período para suportar o volume de dados.
- **Log de auditoria** de todas as mudanças de estado das ordens.

★ **Decisão crítica de modelagem:** o grupo deve pensar como controlar o saldo do usuário entre o momento em que uma ordem é criada e o momento em que ela é executada ou cancelada. Errar aqui compromete toda a consistência do sistema.

2.2 Trigger de Matching (núcleo do sistema)

Ao inserir uma nova ordem, um **Trigger em PL/pgSQL** deve automaticamente:

1. **Buscar contrapartes** elegíveis no order book, respeitando *price-time priority* (melhor preço primeiro; empates resolvidos pelo timestamp mais antigo).
2. **Travar as ordens selecionadas** para execução sem bloquear o restante do book,

evitando deadlocks em ambiente concorrente.

3. **Calcular e executar o volume negociavel**, tratando Partial Fill quando a ordem entrante é maior do que a contraparte disponivel. Por exemplo, eu quero comprar 5 Bitcoins, mas no order book tem alguém vendendo apenas 3 no preço que eu posso comprar. Logo, o sistema compra os 3 bitcoins e aguarda outra ordem de venda aparecer para continuar comprando e executar completamente a minha ordem de compra de 5 bitcoins.
4. **Atualizar saldos, ordens e registrar o trade** de forma totalmente atomica. Qualquer falha deve reverter tudo.
5. **Repetir o loop** ate que a ordem seja totalmente atendida ou nao haja mais contrapartes elegiveis.

A chave da consistência

A combinacao de `SELECT FOR UPDATE SKIP LOCKED`, tratamento de Partial Fill, atualizacao de saldos em ordem deterministica (para evitar deadlock) e rollback condicional. A depuracao exige compreensao real de como o PostgreSQL gerencia transacoes concorrentes.

2.3 Funcoes PL/pgSQL

Alem da funcao de matching, o grupo deve implementar ao menos:

- **Consulta do Order Book:** (*get_best_orders*) retorna os N melhores bids e asks de um ativo em tempo real.
- **Cancelamento de ordem:** (*cancel_order*) cancela uma ordem aberta e restaura o saldo do usuario corretamente.
- **Portfolio do usuario:** (*user_portfolio*) retorna o saldo de cada ativo e o valor total convertido pelo ultimo preco negociado.

2.4 Views

O banco deve expor ao menos as seguintes visões:

- **Resumo de mercado:** (*view_market_summary*) para cada ativo, variacao 24h, volume, ultimo preco e spread atual.
- **Historico de trades:** (*view_trades_history*) lista dos ultimos negocios executados com preco, volume e horario.
- **Ranking de traders:** (*view_traders_ranking*) top negociadores por volume nas ultimas 24h, usando *window functions*.

2.5 Script de Geracao de Dados

O grupo deve desenvolver um script na **linguagem de sua escolha** que popule o banco com dados suficientes para atingir entre **700 MB e 1 GB**. O script deve:

- Simular multiplos traders enviando ordens concorrentemente (ao menos 4 processos/threads paralelos). Você pode simular com 4 terminais abertos.
- Não coloque instruções de *delay()* ou *sleep()* no seu código. O banco de dados

precisa receber uma alta frequência de inserções propositalmente.

- Usar insercao em lote — nao inserts individuais — para atingir o volume exigido em tempo razoavel.
- Gerar precos com variacao realista; a sequencia de precos nao pode ser puramente aleatoria.
- Ao final, imprimir: volume total inserido, numero de trades gerados e tempo de execucao.

3. Entregáveis

1. Relatório técnico (PDF) utilizando o template de artigos¹ da SBC (Sociedade Brasileira de Computação) com as seguintes seções:
 - (a) Membros do projeto e suas respectivas informações de curso, RGA e e-mail.
 - (b) **Justificativa e contexto do problema.** Descreva o problema a ser solucionado e os principais desafios para solucioná-lo.
 - (c) **Modelo ER:** Descreva o modelo entidade e relacionamento desenvolvido. Apresente detalhes de decisões de projeto tomadas nesta fase de construção do banco de dados.
 - (d) **Regras de negócio implementadas e não-implementadas:** Deixe claro quais regras de negócio ou requisitos não foram implementados.
 - (e) **Principais consultas, funções e/ou gatilhos SQL utilizados:** Mostre quais principais consultas estruturas SQL utilizadas e que problemas elas solucionam.
 - (f) **Screenshots das saídas da aplicação:** Apresente os principais relatórios exibidos por sua ferramenta.
 - (g) **Conclusões e melhorias sugeridas:** Apresenta uma conclusão relatando se a sua solução resolve ou não o problema proposto. Além disso, proponha melhorias para o seu código e/ou banco de dados.
2. **Script SQL** de criação e povoamento do banco de dados.
3. **Código-fonte** da aplicação com documentação para execução local.
4. **Apresentação final (15 minutos)** demonstrando o sistema com as regras de negócios e/ou heurísticas implementadas para solucionar o problema escolhido.
5. **TODOS OS ARTEFATOS GERADOS NO ÂMBITO DESTES TRABALHO (CÓDIGO, DIAGRAMAS, VÍDEO, RELATÓRIO, ETC.) DEVEM SER ARMAZENADOS NO GOOGLE DRIVE INSTITUCIONAL DE UM DOS MEMBROS DO GRUPO E COMPARTILHADO COM O PROFESSOR (marcio.inacio@ufms.br).**
6. Apenas um membro do grupo deverá enviar um arquivo texto plano README.MD na área de submissão do AVA contendo o nome completo dos integrantes que de fato fizeram o trabalho, RGAs e o link para o google drive contendo todos os artefatos. A não observância a este item acarretará a nota zero para todos os membros do grupo.

¹<https://www.sbc.org.br/wp-content/uploads/2024/07/modelosparapublicaodeartigos.zip>

7. **Nenhum arquivo do Google Drive poderá ser alterado ou criado após o horário limite de entrega.**
8. Não serão aceitas entregas realizadas por e-mail. Organizem-se para realizar a gravação até algumas horas antes da entrega do trabalho, pois ferramentas como o Google Meet podem entregar o link da gravação até horas depois de realizá-la.

4. Apresentação

Cada grupo deverá gravar um vídeo de no máximo 15 minutos explicando como resolveu o problema proposto. Respondendo os seguintes itens na apresentação:

- Apresente-se falando o nome completo e qual curso cada integrante do grupo pertence;
- Apresentar as tabelas e seus respectivos atributos;
- Como é feita a entrada de dados para o PostgreSQL?
- Fizeram algum script/programa para importar os dados?
- Qual linguagem escolheram? Por que esta linguagem?
- Mostre o script/programa interagindo com o banco de dados vazio.
- Explique se os requisitos foram totalmente atendidos ou parcialmente. Se parcialmente, diga quais foram os itens atendidos.

A apresentação é um item indispensável para obter a nota do trabalho. Os alunos que não apresentarem terão sua nota final do trabalho composta apenas pela nota geral do grupo, obtida pela entrega do banco de dados e eventuais códigos.

O vídeo não pode ter cortes ou edições que interrompam o fluxo contínuo da apresentação. Softwares como Google Meet, Zoom.us, Skype podem ser úteis para a gravação. O Zoom gratuito permite que você grave a reunião localmente.

5. Considerações Finais

- **O banco de dados para o desenvolvimento deste trabalho deve ser OBRIGATORIAMENTE o PostgreSQL 15+ ou superior.**
- **O MENOR INDÍCIO DE USO DE LLMs (ChatGPT, Claude AI, Gemini e similares) EM QUALQUER FASE DESTA TRABALHO, ACARRETARÁ ZERO PARA TODOS OS ENVOLVIDOS.**
- Não serão aceitos trabalhos atrasados. Se o grupo não entregar o trabalho no dia combinado, ele receberá nota zero.
- Em caso de projetos copiados de colegas todos os envolvidos recebem nota zero. Lembre-se é muito improvável que haja trabalhos totalmente iguais ou grau de similaridade muito alta.
- O professor não ajudará os grupos na construção do trabalho.
- O professor poderá tirar dúvidas conceituais em horário de aula ou horário de atendimento.
- A nota dos integrantes não necessariamente será a mesma. Se durante a apresen-

tação o professor detectar que algum integrante do grupo não tem domínio sobre o projeto, ele poderá receber uma nota menor que os demais integrantes.

- Entrevistas presenciais podem ser solicitadas a determinados grupos a fim de elucidar eventuais dúvidas/suspeitas do professor.

6. Cálculo da Nota Final com Penalidade por Ausência na Apresentação

Tabela 1: Critérios de Avaliação do Projeto

	Peso
Modelagem de dados e normalização	2,0
Resolução do problema proposto	2,5
Banco de Dados em SQL (com dados)	1,5
Funcionalidade e usabilidade do script ou aplicação.	1,5
Documentação técnica clara e bem estruturada	1,0
Apresentação final e domínio do conteúdo	1,5
Total (N)	10,0

Seja:

- N : nota total obtida com base nos critérios avaliativos (valor entre 0 e 10);
- A : variável binária indicando a presença na apresentação final, definida como:

$$A = \begin{cases} 1, & \text{se o aluno participou da apresentação final} \\ 0, & \text{se o aluno não participou da apresentação final} \end{cases}$$

A nota final N_f será calculada da seguinte forma:

$$N_f = N \cdot (0,5 + 0,5 \cdot A)$$

Exemplos

- Se o aluno participou da apresentação ($A = 1$):

$$N_f = N \cdot (0,5 + 0,5 \cdot 1) = N \cdot 1 = N$$

- Se o aluno não participou da apresentação ($A = 0$):

$$N_f = N \cdot (0,5 + 0,5 \cdot 0) = N \cdot 0,5$$

Essa regra tem como objetivo incentivar a participação ativa na apresentação final do projeto, que é parte fundamental do processo avaliativo.